

TOWER SIEGE

A tower-defense game built on C++ data structures & algorithms

Daniel Santiago Pérez Madera

Code 20231020203

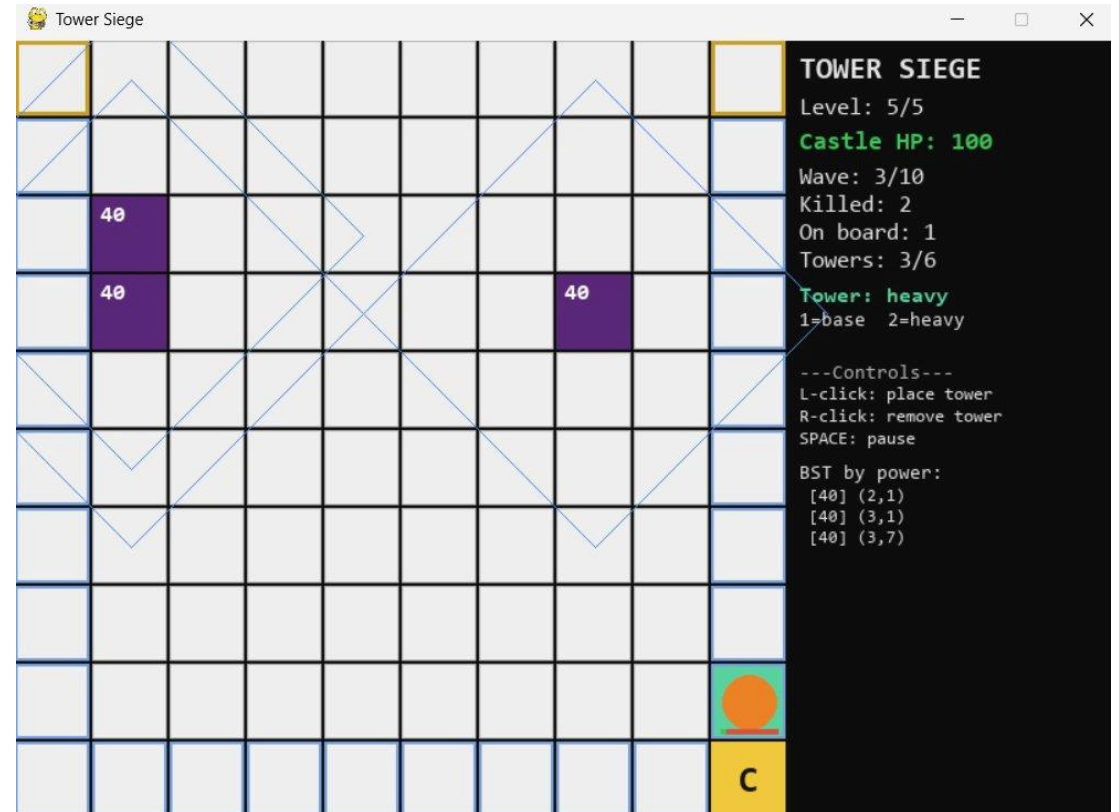
Computer Science I · Season 2026-I

Universidad Distrital Francisco José de Caldas · Bogotá, Colombia

What is Tower Siege?

A grid-based tower-defense game. Enemies spawn and march toward the castle; the player places towers to stop them.

- Runs locally on the desktop.
- C++ engine drives all game logic.
- Pygame renders the grid, towers and HUD.
- Two tower types: base and heavy.



Live gameplay — towers (purple), enemy spawn, HUD with stats.

System Architecture

Pygame UI Layer

grid rendering · input · HUD



C++ Engine — Linked Lists

enemies & towers on the board



BST — Tower ranking by power

ordered by damage value



Greedy + Backtracking

placement advice & pathfinding

Data Structures Used

Singly Linked Lists

Manage the dynamic sets of objects on the board.

- Active enemies marching to the castle.
- Towers placed by the player.
- Insertions and removals via raw pointers.

Binary Search Tree (BST)

Ranks towers by their power value.

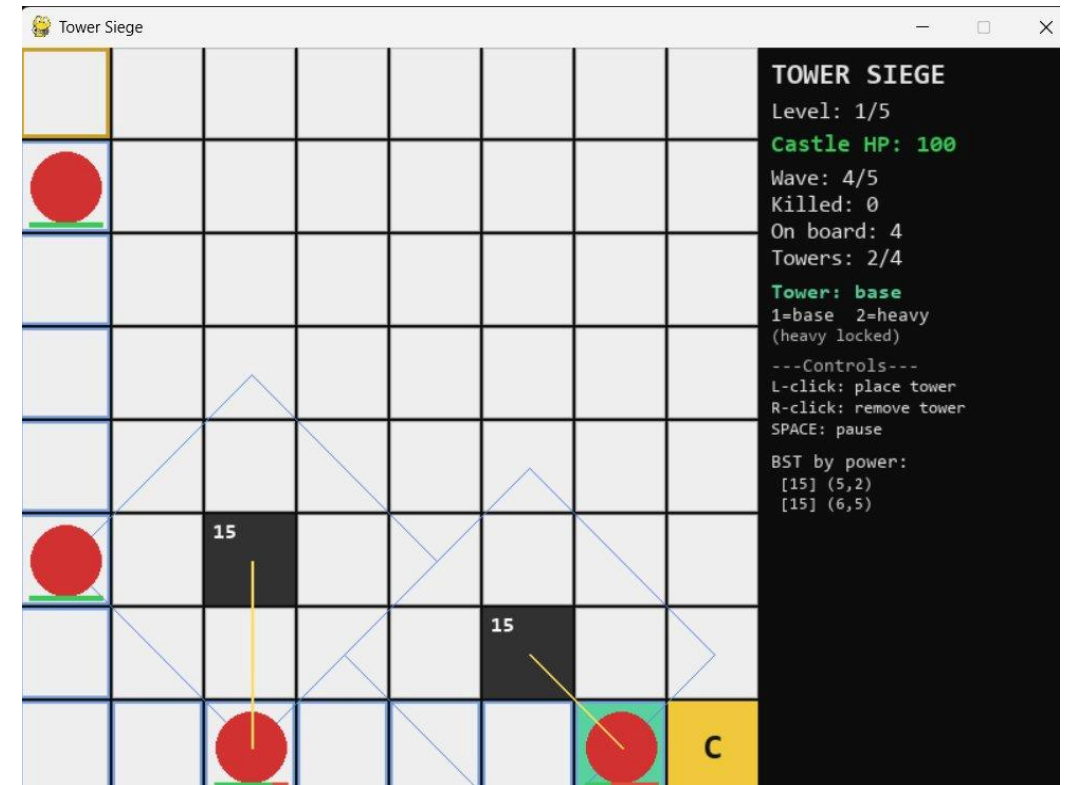
- Ordered traversal lists towers by strength.
- Shown live in the HUD: [40] (2,1), [15] (6,5)...
- Supports fast lookup of the strongest tower.

Algorithm 1 – Greedy

Tower placement advisor

At each step, Greedy picks the locally best spot to place a tower — the cell that maximizes coverage right now.

- Always favors cells close to the castle.
- Fast: no look-ahead, just the best immediate choice.
- **Optimal locally — but blind to global consequences.**



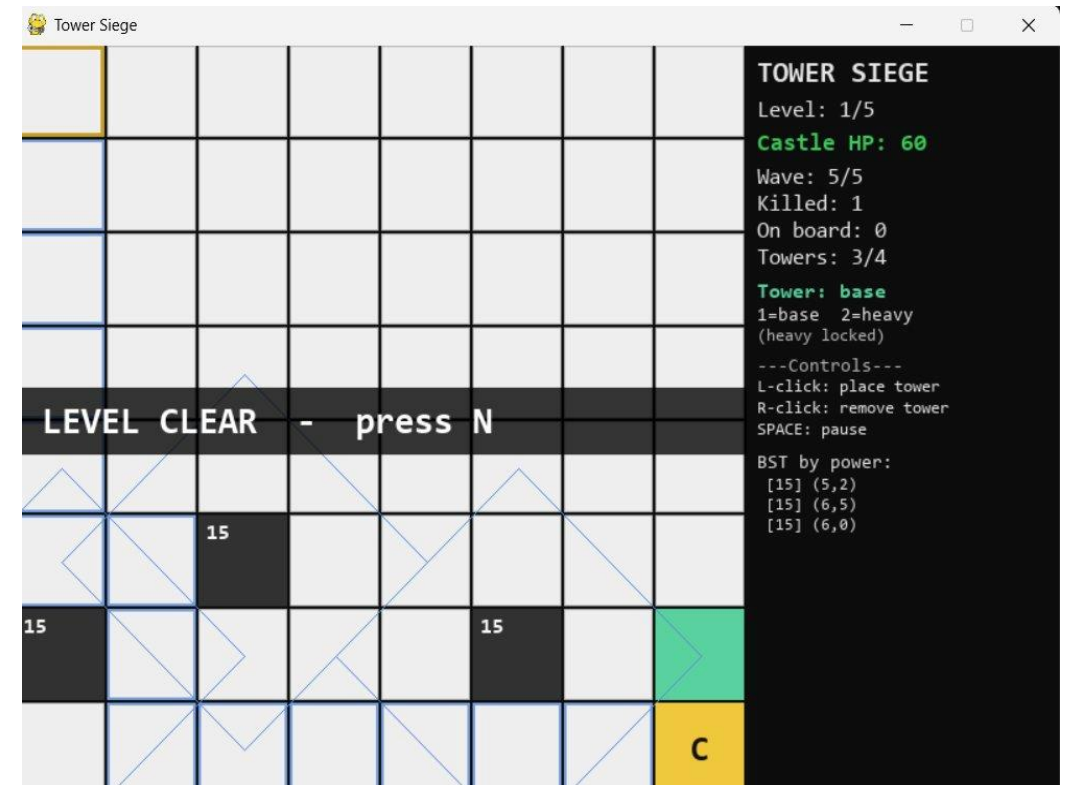
Towers positioned to cover incoming enemy paths.

Algorithm 2 – Backtracking

Enemy pathfinding

Backtracking explores possible routes from the enemy spawn point to the castle, searching for the fastest path around the towers.

- Tries a route; if blocked, steps back and tries another.
- Guarantees a valid path if one exists.
- **Cost grows exponentially as paths get blocked.**



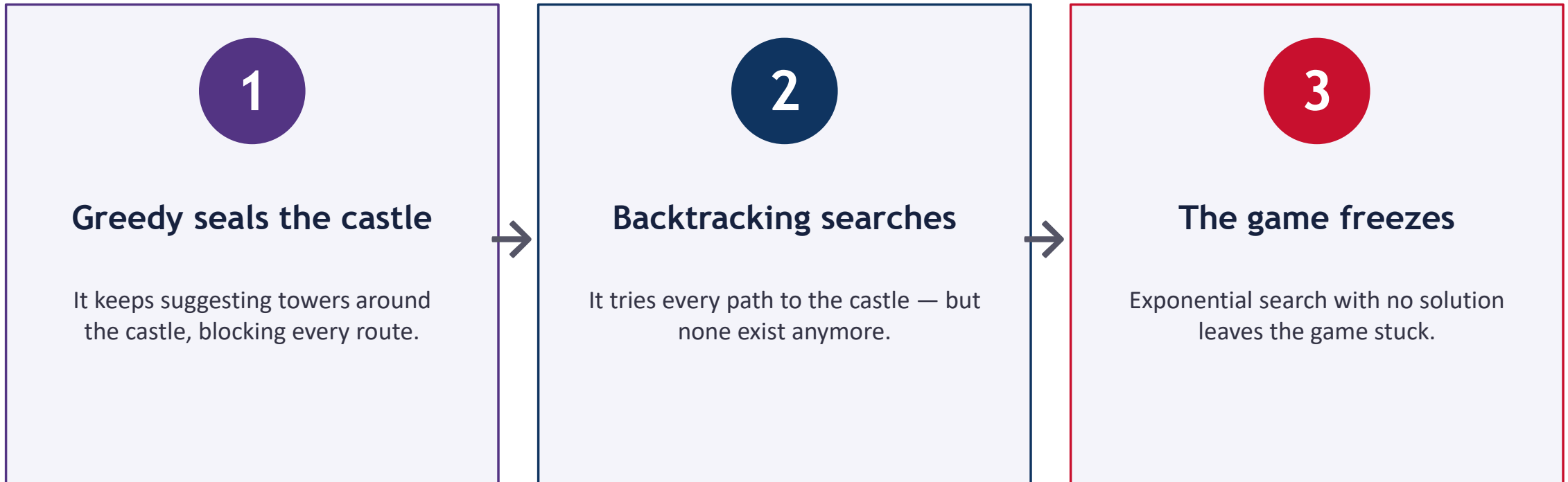
Enemies find their way along the open grid routes.

Main Technical Challenge: Pointers

Building everything on raw C++ pointers — with no STL containers or external libraries — was the hardest part of the project.

- Dangling pointers when removing dead enemies mid-wave.
- Keeping list and BST references consistent after each update.
- Manual memory management to avoid leaks across waves.

The Algorithm Conflict



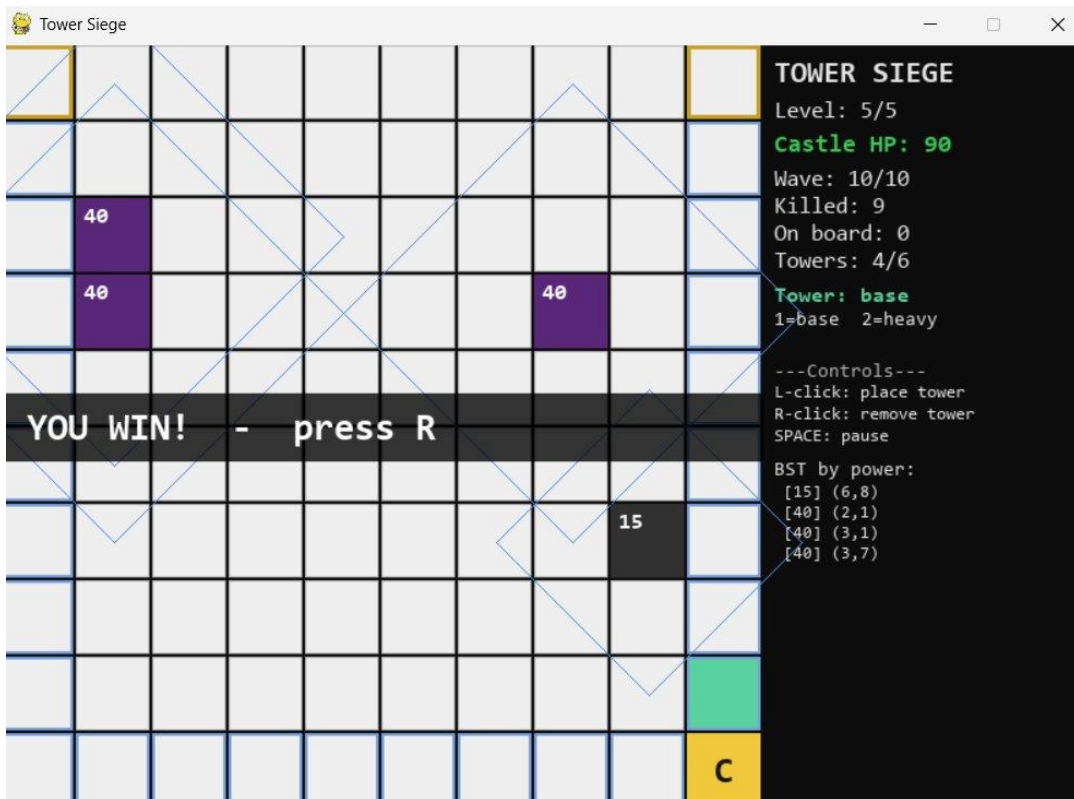
Two locally-correct algorithms can conflict globally.

Testing Strategy

Unit tests on each data structure plus integration tests on the full game loop — checking pointer integrity, BST ordering, and algorithm output.

Test	Focus	Result
Linked list insert / remove	Pointer integrity	PASS
BST insert by power	Ordered ranking	PASS
Greedy tower suggestion	Best placement	PASS
Backtracking pathfinding	Fastest route	PASS
Greedy + Backtracking combined	Algorithm cooperation	FREEZE

Results



All five levels are playable and winnable.

Strengths

- Runs fully on hand-built data structures.
- All five levels playable.
- Both algorithms correct in isolation.

Weakness

- Greedy can seal all routes, sending Backtracking into an exponential dead-end search that freezes the game.

Conclusions

The research question is answered: yes — a tower-defense game can be built purely on custom C++ data structures, and it makes Greedy and Backtracking tangible for learners.

The freeze taught the deepest lesson: two locally-optimal algorithms can break each other globally.

Future work

- Constrain Greedy so it never fully seals a path.
- Give Backtracking a depth limit and a fallback route.

Thank you

Questions discussion

Daniel Santiago Pérez Madera · Code 20231020203
github.com/dspm2212/Tower_Siege

References: Pygame docs · nlohmann/json · Python json module